

A Holistic and Hybrid Service Selection Strategy for MEC-Based UAV Last-Mile Delivery Systems

Jia Xu , *Member, IEEE*, Xiao Liu , *Senior Member, IEEE*, Azadeh Ghari Neiat , Liju Chu ,
Xuejun Li , *Member, IEEE*, and Yun Yang , *Senior Member, IEEE*

I. INTRODUCTION

Abstract—With the widespread use of Internet of Things (IoT) technology, an enormous number of end devices that request various kinds of cloud services have been connected to the Internet. Multi-access edge computing (MEC) can reduce the service response time by selecting the required edge computing resources closer to the end device. However, MEC-based smart systems require heterogeneous and diverse services support. Taking unmanned aerial vehicle (UAV) last-mile delivery system as an example, there are two types of services required: delivery and computational services. The edge services in MEC environments are distributed and limited. Inefficient service selection plans will affect the quality of services of such smart systems. Therefore, how to design a suitable service selection strategy is a crucial issue for MEC-based smart systems. To address this issue, we propose a service selection framework and a holistic and hybrid service selection (H^2S^2) strategy for MEC-based UAV last-mile delivery systems in real-world UAV last-mile delivery scenarios. This framework considers three important characteristics of UAV delivery systems: diverse service requirements, service availability, and service mobility. The H^2S^2 strategy focuses on selecting the optimal delivery and computational services and provides an integrated approach with a static service selection algorithm and a dynamic service re-selection algorithm. The H^2S^2 strategy determines the optimal delivery and computational service selection plans with the lowest UAV energy consumption and shortest service response time. We assess the effectiveness and efficiency of the H^2S^2 strategy through ablation studies and comparative analyses with diverse representative strategies. The experimental results show that the H^2S^2 strategy improves the effectiveness and efficiency of the UAV delivery system by significantly reducing UAV's energy consumption and service response time.

Index Terms—Multi-access edge computing, service selection, smart logistics systems, UAV delivery.

Received 18 January 2024; revised 3 July 2024; accepted 19 August 2024. Date of publication 28 August 2024; date of current version 30 December 2024. This work was supported in part by Young Scientists Fund of the National Natural Science Foundation of China under Grant 62302005, in part by China Postdoctoral Science Foundation under Grant 2023M740014 and in part by Anhui Provincial Natural Science Foundation, China, under Grant 2308085QF219. (Corresponding authors: Xiao Liu; Xuejun Li.)

Jia Xu is with the School of Internet, Anhui University, Heifei, Anhui 230093, China (e-mail: xujia@ahu.edu.cn).

Xiao Liu and Azadeh Ghari Neiat are with the School of Information Technology, Deakin University, Geelong, VIC 3220, Australia (e-mail: xiao.liu@deakin.edu.au; azadeh.gharineiat@deakin.edu.au).

Liju Chu and Xuejun Li are with the School of Computer Science and Technology, Anhui University, Heifei, Anhui 230093, China (e-mail: clj@stu.ahu.edu.cn; xjli@ahu.edu.cn).

Yun Yang is with the Department of Computing Technologies, Swinburne University of Technology, Melbourne, VIC 3122, Australia (e-mail: yyang@swin.edu.au).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSC.2024.3451243>, provided by the authors.

Digital Object Identifier 10.1109/TSC.2024.3451243

IN RECENT years, the Internet of Things (IoT) has led to the development of various smart systems and applications, including smart logistics, smart homes, smart health, smart agriculture and smart transportation [1], [2]. As a result, the number of smart end devices connected to the Internet has increased drastically. According to Statista, the number of connected smart end devices worldwide will increase to 45.72 billions by 2028 [3]. This means that nearly 46 billions end devices will need to access the Internet and request various kinds of services. Currently, with the help of cloud computing-based smart systems and applications, end devices can easily access complex and diverse cloud services. However, the rapidly increasing number of service requests generated by end devices could cause challenges, including network congestion and conflicts. Therefore, cloud services could face problems such as high service response times, low system efficiency, and an inability to meet the service quality requirements of users [4], [5].

Multi-access edge computing (MEC) has been developed to address the challenges faced by cloud computing [2], [6]. MEC has revolutionized the cloud computing architecture by moving cloud services and resources to be near end devices in edge servers. Combined with 5G and smart systems, MEC can offer end devices or users a better Quality of Service (QoS) and Quality of Experience (QoE). In particular, in delay-sensitive smart systems, the service response time of the end device can be effectively reduced because the end device is closer to the edge servers than the cloud servers. As a result, the MEC environment has been widely used in various kinds of smart systems, such as smart logistics, smart health and smart transportation. However, edge servers have limited resources, and they may not satisfy the service requirements of users or may even significantly degrade the QoS of the smart system. Therefore, how to design a suitable service selection strategy is a crucial issue for MEC-based smart systems.

Last-mile delivery is a critical component of smart logistics systems due to its significant impact on the overall logistics cost and time. It typically accounts for a substantial portion of the total delivery cost and time. For example, according to the report by CapGemini,¹ last-mile delivery constitutes 53% of total shipping costs and 41% of overall supply chain costs. Efficient last-mile delivery solutions are therefore essential for optimizing logistics operations and enhancing customer satisfaction. UAV deliveries significantly enhance accessibility for individuals with limited transportation options and services. They are also

¹[Online]. Available: <https://www.capgemini.com/insights/expert-perspectives/navigating-the-complex-web-of-last-mile-deliveries/>

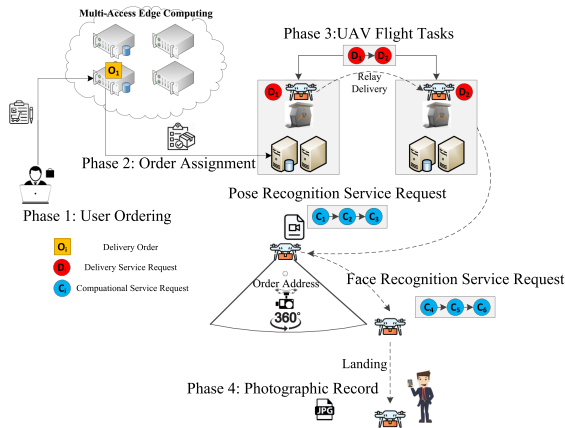


Fig. 1. Typical UAV delivery process.

more cost-effective than traditional express delivery methods, costing approximately \$1.23 per delivery compared to \$5.33 for last-mile distance by electric van [7]. This cost-efficiency positions UAV to serve a niche market for expedited goods delivery, ushering in a new era of sustainable smart logistics systems.

In the process of UAV delivery, there are many types of applications and services that need to be used, such as delivery services and computational services. The execution of such services requires significant energy consumption of UAVs. However, due to the size and capacity of the batteries, UAVs often cannot complete the entire delivery process independently. Therefore, the UAVs need to request different types of services to complete the delivery process. Last-mile delivery is typically suited for small-scale deliveries within urban areas, where the environment is relatively complex and requires real-time detection and planning adjustments. In contrast, long-distance delivery is more applicable to rural and mountainous regions. This complexity in urban areas makes the joint consideration and optimization of delivery and computational service resources a significant challenge. Additionally, urban environments tend to have higher densities of UAV delivery operations, increasing the likelihood of resource conflicts compared to long-distance deliveries. To address this, it is crucial to dynamically adjust the selection of delivery and computational services based on the actual delivery process. This helps avoid service conflicts and ensures the efficiency of the delivery operations.

As shown in Fig. 1, the typical UAV delivery process can be divided into four main phases. Phase 1 involves user ordering, where users submit logistics and transportation service demands within the MEC environment. Phase 2, i.e., as order assignment, involves assigning the most suitable delivery service resources to fulfill user orders. This phase ensures optimal allocation to efficiently meet user requirements. Phase 3 encompasses UAV flight tasks, requiring real-time computation for various services during UAV flight, such as target detection and obstacle avoidance. The final phase, Phase 4, focuses on image documentation, for photographic record. During this stage, user identity verification is conducted, and the entire delivery process is recorded for documentation purposes.

In the delivery process, the UAVs need to request two kinds of services: delivery services and computational services. The delivery services are responsible for handling the UAV's delivery requirements, such as delivery task assignments and delivery

relays [8]. The computational services are responsible for handling all types of computational tasks during the UAV's flight process. UAVs often generate a vast amount of data during their operations, such as location information, sensor readings, and images. Computational services can be utilized to process this data in real time, enabling tasks such as object recognition, path optimization, and collision avoidance. [9]. The execution of these functions or tasks necessitates computational capabilities situated at the network's edge. For example, when the delivery process reaches Phase 3 (UAV Flight Tasks), the UAV needs to request a relay delivery service and a computational service to finish computational tasks such as pose and face recognition. This study focuses on UAV delivery scenarios with relative independence between services requests. This allows us to simplify the optimization problem and develop initial solutions.

Existing studies have mostly focused on the service selection optimization problem and have proposed different algorithms for determining the optimal service selection plan [10], [11], [12], [13], [14]. Moreover, most works have considered the perspective of service providers, optimizing the service selection plan with less focus on the perspective of service users [15], [16]. From the user's perspective, there are three important factors that should be considered: the mobility of the user's end device, the dynamic nature of the network environment and the diversity of the user's QoS [17], [18], [19]. Most research works have considered only one or two of these factors [14], [18], and very few studies have considered all of these factors. However, many real-world smart systems, such as MEC-based UAV last-mile delivery systems, must consider these complicated characteristics at the same time. An unsuitable service selection plan may increase the service response time and UAV's energy consumption, which can impact the efficiency of delivery system and cause delivery task failures. Hence, in this research paper, we explore this matter from the standpoint of service consumers (namely, UAV delivery companies) and propose a holistic and hybrid service selection strategy (H^2S^2) for MEC-based UAV last-mile delivery systems. This paper makes the following primary contributions.

1) We introduce a novel service selection framework for MEC-based UAV delivery systems grounded in a real-world scenario. Traditionally, the delivery service and computational service are optimized independently. In contrast, our proposed framework unifies the two services for collaborative optimization, which enhances the efficiency and effectiveness of the delivery system by concurrently reducing the UAV's energy consumption simultaneously.

2) We introduce two new models (viz. the UAV service delivery model and the UAV computational service model) for optimizing the UAV's energy consumption under the order's deadline. The two new models take care of three aspects: diverse service requirements, service availability and mobility, which can effectively address the issues of the conflict and availability of services in UAV delivery.

3) We propose a holistic and hybrid service selection strategy for MEC-based smart logistics systems that can optimize the multi-UAV service selection problem. The proposed strategy consists of a set of algorithms which cover the entire UAV delivery process from a holistic optimization perspective. Meanwhile, existing works often deal with the static and dynamic service selection problems independently which cannot guarantee a global optimization. In contrast, our proposed hybrid strategy incorporates both the static and dynamic service selection with

the aim to determine the optimal delivery and computational service selection plans.

4) We conduct comprehensive experiments using Antwork's ADNET scenario to validate the effectiveness and efficiency of H^2S^2 strategy. The simulation experiment dataset was abstracted from real UAV air path data collected by the ADNET project. The experimental results indicate that H^2S^2 strategy achieves an average 9.71% improvement in delivery efficiency and 15.42% reduction in delivery energy consumption for improvement of effectiveness when the number of orders is 6.

The subsequent sections of the paper are structured as follows. Section II presents the delivery and computational service models according to real-world scenarios of last-mile delivery utilizing MEC and UAV. (Appendix A.1, available online²). Section III addresses the system framework, system model and problem formulation for delivery service and computational service selection using the problem analysis of the last-mile delivery scenarios (Appendix A.2, available online²). Section IV elaborates on the H^2S^2 strategy. Section V provides performance evaluations of the H^2S^2 strategy. Section VI reviews pertinent literature on service selection algorithms within the domains of cloud computing and MEC environment. In conclusion, Section VII summarizes the findings of this paper and outlines avenues for future research.

II. SERVICE MODELS

Within this section, two types of service models are defined and described for MEC-based UAV last-mile delivery systems: delivery and computational service models. We summarize the notations introduced in this section and used in this paper in Appendix B, available online.²

Definition 1: Delivery service request

For delivery order O_i , the delivery service request DR_j is expressed as a 5-tuple (RID^d, W, SP, DP, DL^d) , where

- 1) RID^d is the distinctive identifier of delivery service request DR_j ;
- 2) W is the weight of package for O_i ;
- 3) SP is the starting point of O_i ;
- 4) DP is the destination point of O_i ;
- 5) DL is the deadline of O_i .

In MEC-based UAV last-mile delivery systems, because of the limited delivery capacity of the UAV, the delivery order O_i may request multiple delivery services, which form a UAV relay delivery request set $\mathbf{DR} = \{DR_1, DR_2, \dots, DR_n\}$. Each delivery service request DR_j is executed on a delivery service instance, which is defined below.

Definition 2: Computational service request

For UAV U_i , the computational service request CR_j is expressed as a 5-tuple $(RID^c, DI, DO, WL, DL^c)$, where

- 1) RID^c denotes the distinctive identifier of CR_j ;
- 2) DI denotes the input data size of CR_j ;
- 3) DO denotes the output data size of CR_j ;
- 4) WL denotes the workload of CR_j ;
- 5) DL^c denotes the deadline of CR_j .

When the UAV delivers the package from the starting point to the destination, it uses various computational service requests. These service requests form the computational service request

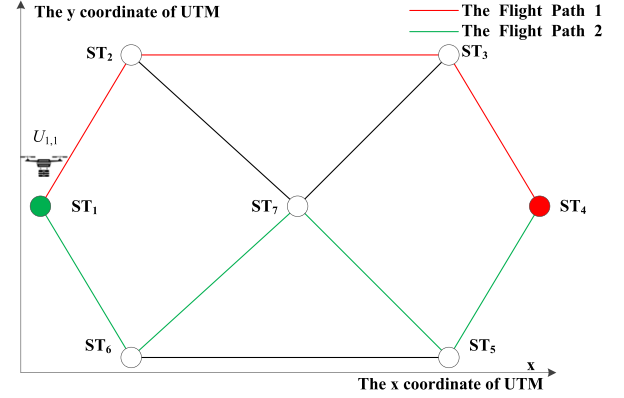


Fig. 2. The movement path of the UAV in MEC-based last-mile delivery systems.

set $\mathbf{CR} = \{CR_1, CR_2, \dots, CR_n\}$. Each computational service request CR_j is executed on a computational service instance, which is defined below.

Definition 3: Delivery service model

A delivery service $DS_{i,j}$ is represented as a 6-tuple $(ID^d, RID^d, DS^S, DS^D, U_{i,j}, FR_{i,j})$, where

- 1) ID^d is the distinctive identifier of delivery service plan;
- 2) RID^d is the distinctive identifier for the service request associated with the delivery service plan;
- 3) DS^S is the delivery station at the starting point;
- 4) DS^D is the delivery station at the destination;
- 5) $U_{i,j}$ is the delivery UAV at DS^S . For each delivery UAV, $U_{i,j} = \{ML_{i,j}, MS_{i,j}, BC_{i,j}, \sim FP_{i,j}\}$ where $ML_{i,j}$ is the maximum payload of $U_{i,j}$, $MS_{i,j}$ is the maximum speed of $U_{i,j}$, $BC_{i,j}$ is the battery capacity of $U_{i,j}$, and $FP_{i,j}$ is flight power of $U_{i,j}$.
- 6) $FR_{i,j}$ is the flight path of $U_{i,j}$.

Definition 4: Computational service model

A computational service $CS_{i,j}$ is represented by a 3-tuple (ID^c, RID^c, SP, F) , where

- 1) ID^c is the distinctive identifier of computational service plan;
- 2) RID^c is the distinctive identifier of service request corresponding to the computational service plan;
- 3) SP is the computational service provider. If the computational service is provided by an edge server node, SP represents the edge server node's number. If the computational service is provided by a cloud centre, SP represents the virtual machine's number in the cloud server.
- 4) F is the service processing speed (represented by the CPU frequency) of the service provider.

Definition 5: UAV movement path

The movement path for MEC-based UAV last-mile delivery systems is illustrated in Fig. 2. The start and destination points of UAV $U_{1,1}$ are ST_1 and ST_4 , respectively. There are two possible flight paths: ST_1 - ST_2 - ST_3 - ST_4 and ST_1 - ST_6 - ST_7 - ST_5 - ST_4 . In accordance with the designated paths, we establish a two-dimensional Cartesian coordinate system utilizing the universal transverse mercator method to convert latitude and longitude information of the delivery station and UAV location coordinates into Cartesian coordinates [20]. The UAV's flight path is predetermined by the air traffic management office and is depicted by the line connecting the delivery stations in Fig. 2. The UAV,

²Appendix access URL: <https://drive.google.com/file/d/1NghUCtd3k3pRrG4Wq74719tyh1ozULB1/view?usp=sharing>

carrying its package, initiates its flight from the departure point (green point) and proceeds towards the destination point (red point) following the specified flight paths. For example, the distance of the ST_1 - ST_2 flight path can be calculated as follows:

$$|\vec{SD}| = |\vec{T_1T_2}| = \sqrt{(x^{T_1} - x^{T_2})^2 + (y^{T_1} - y^{T_2})^2} \quad (1)$$

where x^{T_1} and x^{T_2} are the x -axis coordinates of the starting point ST_1 and destination point ST_2 , respectively. y^{T_1} and y^{T_2} are the y -axis coordinates of the starting point ST_1 and destination point ST_2 , respectively.

Definition 6: Delivery service selection plan

During the process of UAV delivery, there are many practical constraints such as infeasible paths due to obstacles, availability of UAV airlines due to traffic control, weather factors, and so on. However, we did not consider these practical constraints in this paper. Therefore, this paper considers two important indicators for delivery service selection plan which is the UAV's delivery time and energy consumption. The calculation methods for these indicators are shown below.

1) $DT_{i,j}$ is the delivery time when UAV $U_{i,j}$ follows flight path FR , which is calculated as

$$DT_{i,j} = FT_{i,j} + RT_{i,j} \quad (2)$$

$$FT_{i,j} = \frac{|\vec{SD}|}{MS_{i,j}} \quad (3)$$

where $FT_{i,j}$ and $RT_{i,j}$ are the flight time and relay time of $U_{i,j}$, $\vec{SD}$ is the distance of the flight path FR , and $MS_{i,j}$ is the maximum speed of $U_{i,j}$.

2) $DE_{i,j}$ is the delivery energy consumption of UAV $U_{i,j}$, which can be calculated as follows:

$$DE_{i,j} = FE_{i,j} + RE_{i,j} \quad (4)$$

$$FE_{i,j} = FP_{i,j} \times FT_{i,j} \quad (5)$$

where $FE_{i,j}$ and $RE_{i,j}$ are the flight energy consumption and relay energy consumption of $U_{i,j}$ respectively, $FP_{i,j}$ is the flight power of $U_{i,j}$ and $FT_{i,j}$ is the flight time of UAV $U_{i,j}$ [9], [21]. The flight power of n-rotor UAV $FP_{i,j}$ can be calculated as follows [22]:

$$FP_{i,j} = (FW_{i,j} + W_{i,j} + w_i)^{\frac{3}{2}} \sqrt{\frac{g^3}{2\rho\zeta n}} \quad (6)$$

where $FW_{i,j}$ is the frame weight of UAV $U_{i,j}$, $W_{i,j}$ is the battery weight of UAV $U_{i,j}$, w_i is the payload weight of the package for delivery order O_i , g stands for acceleration of gravity, ρ is fluid density of air, ζ is the area of the rotating blade.

Definition 7: Computational service selection plan

The computational service selection plan has three important indicators.

1) $CT_{i,j}$ denotes the service response time of computational service $CS_{i,j}$, and it can be calculated as follows:

$$CT_{i,j} = CP_{i,j} + TT_{i,j} \quad (7)$$

The total service response time is partitioned into two components. The initial segment is the processing time of the computational service $CP_{i,j}$. The subsequent segment is the data transmission time $TT_{i,j}$ between the UAV and the service

provider.

$$CP_{i,j} = \frac{WL_{i,j}}{F_{i,j}} \quad (8)$$

$$TT_{i,j} = \frac{DI_{i,j}}{R(tp_{i,j}, LT)} + \frac{DO_{i,j}}{R(rp_{i,j}, LR)} \quad (9)$$

where $WL_{i,j}$ is the workload of computational service request CR_j in delivery order O_i ; $F_{i,j}$ is the service processing speed of the service provider; $DI_{i,j}$ and $DO_{i,j}$ represent the size of the input data and output data for service request CR_j in delivery order O_i , respectively; $R(tp_{i,j}, LT)$ and $R(rp_{i,j}, LR)$ are data transfer speed and data reception speed, respectively; and $tp_{i,j}$ and $rp_{i,j}$ are the transfer and reception power of UAV $U_{i,j}$, respectively. The details of the wireless transmission speed can be found in [23], [24].

2) MR is the deadline miss rate of computation service requests, which can be calculated as follows:

$$MR = \frac{SN_D}{SN_T} \quad (10)$$

where SN_D denotes the count of computational service requests that failed to meet their deadlines, and SN_T stands for the total number of computational service requests.

III. SYSTEM FRAMEWORK AND PROBLEM FORMULATION

This section introduces the system framework for service selection in MEC-based UAV last-mile delivery systems. Unlike the system models proposed in the previous section, this system framework focuses on the design of the service management module in MEC-based UAV last-mile delivery systems and provides the basis for modelling the problem. Subsequently, we formulate the service selection problem within the context of MEC-based UAV last-mile delivery systems, building upon the proposed service selection framework.

A. System Framework

Drawing upon the motivating example and the problem analysis outlined in Appendix A, available online,² we establish a service selection framework tailored for MEC-based UAV last-mile delivery systems. As shown in Fig. 3, the service selection framework considers two kinds of services requested by the UAV: delivery services and computational services. The delivery service and computational service resources in the MEC environment can be efficiently managed by this system framework.

The service selection framework is divided into five layers: infrastructure layer, virtualization layer, service function layer, service management layer and application layer. The application layer receives the two types of service requests from the UAV: delivery service requests and computational service requests. The service management layer analyzes the service requests received from the application layer. Then, the best service selection plan is generated by the service management controller. When the service function layer receives the service selection plans from the above layer, it invokes the available service functions in this layer to execute the UAV's service requests. The virtualization layer provides a unified abstraction of the bottom infrastructure layer, which includes the computational service function (CSF), delivery service function (DSF) and network virtualization. The

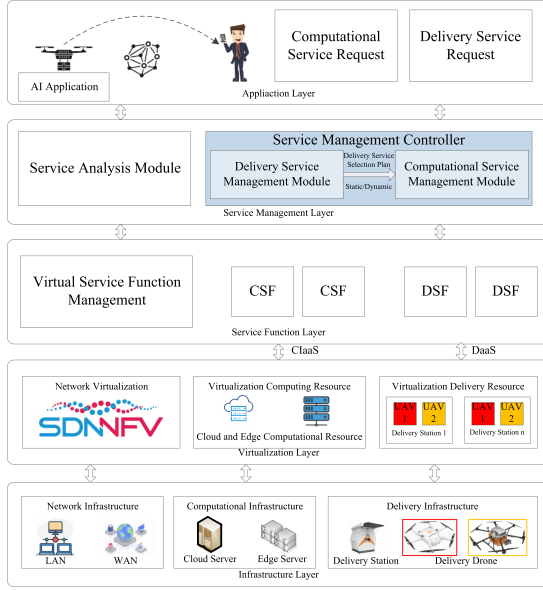


Fig. 3. The service selection framework for MEC-based UAV last-mile delivery systems.

foundational layer is the infrastructure layer, encompassing network infrastructure, computational infrastructure, and delivery infrastructure.

The above analysis shows how a suitable service selection plan can be generated according to the characteristics of the UAV's service requests which is a crucial issue in ensuring the efficiency of MEC-based UAV last-mile delivery systems. Therefore, suitable algorithms should be designed for the computational and delivery service management sub-modules in the service management controller (highlighted in a slate blue color). In this paper, the H^2S^2 strategy focuses on the delivery and computational service selection optimization problems, providing an integrated approach that considers both the delivery and computational service selection algorithms and the dynamic service re-selection algorithm.

B. System Models

The system model contains formal definitions of various types of services as well as the definitions and calculation methods for optimization metrics in the MEC environment. As shown in Fig. 4, the system model of the MEC-based UAV last-mile delivery systems has four components: UAVs, edge service resources, cloud service resources and a service management controller module [25], [26].

The UAVs in MEC-based UAV last-mile delivery systems are the end devices that deliver the user's package (represented by U_i). The delivery process can be divided into two main stages: the static stage and the dynamic stage. In the static stage, the UAVs at each delivery station are in standby states and wait for instructions from the delivery system. After the delivery instructions are issued by the system, the UAV flies to the destination delivery station. In this process, because of battery and computing capacity limitations, a UAV cannot handle the service requests by itself. Therefore, the UAV needs to submit requests to an edge or cloud service. As shown in Fig. 4, UAV U_1 requests edge service R_1 from CS_4^E . After the service request is

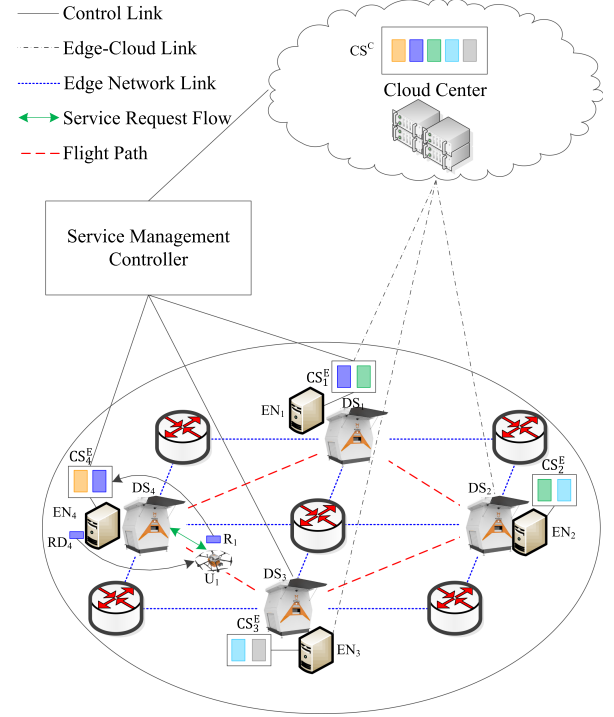


Fig. 4. The system model of the MEC-based UAV last-mile delivery systems.

processed, the resulting data RD_4 is sent back to U_1 . Note that these service requests are independent of each other.

The edge service resources consist of two types of service resources: delivery services and computational services. The delivery service is mainly composed of various types of UAVs and delivery stations. First, when delivery orders are sent to the starting delivery station, the most suitable UAV should be selected for the packages. Then, the best flight path between the starting and destination points should be planned before the UAV takes off. Finally, during the delivery process, if the current UAV cannot deliver the cargo to the destination, the best relay delivery plan should be determined to continue the delivery process. As shown in Fig. 4, the service management controller allocates U_1 to deliver package O_1 from delivery station DS_3 to DS_1 . The best flight path is DS_3 - DS_4 - DS_1 . During the process of U_1 delivering the package, relay UAV U_2 is required for relay delivery. The edge computational services are located in various edge server nodes at the delivery station. The edge service nodes are connected to each other and provide limited computational services to the UAVs in the area. For instance, there are four edge server nodes in Fig. 4, represented by EN_1 , EN_2 , EN_3 and EN_4 , and U_1 requests the edge computational service R_1 from EN_4 .

Cloud service resources offer powerful capabilities and virtually unlimited computational resources for UAVs. As shown in Fig. 4, the cloud servers are represented by CS^C . Because of the long distance between the cloud servers and the UAV, the data transfer time of cloud services is usually higher than that of edge services. Therefore, cloud services are only suitable for computational resource-intensive service requests.

The service management controller module provides global delivery and computational service control and management functions for MEC-based UAV last-mile delivery systems. When the UAV sends a service request to the system, the service

management controller module considers both the UAV's requirements and the current MEC service environment when selecting the best service. Therefore, how to design an effective and efficient method to optimize the service selection plan is a key challenge.

C. Problem Formulation

Leveraging the proposed service selection framework, we initiate an analysis of the service selection problem within MEC-based UAV last-mile delivery systems, including both the delivery service management problem and the computational service management problem. Then, a series of problem models are defined to formulate the service selection problem with optimization objectives.

Definition 7: Delivery service optimization problem

1) The total delivery time can be calculated as

$$DT_{\text{sum}} = \sum_{i=1}^{N_O} \sum_{j=1}^{N_U} DT_{i,j} \quad (11)$$

For delivery order set $\mathbf{O} = \{O_1, O_2, \dots, O_{N_O}\}$, the total delivery time is the aggregate of all delivery times $DT_{i,j}$ for UAV $U_{i,j}$.

2) The total delivery energy consumption can be formulated as:

$$DE_{\text{sum}} = \sum_{i=1}^{N_O} \sum_{j=1}^{N_U} DE_{i,j} \quad (12)$$

The total delivery energy consumption is the aggregate of all delivery energy consumption $DE_{i,j}$ for UAV $U_{i,j}$.

3) Optimization objective of the delivery service selection problem: $FIUV_1$. The objective is to optimize energy consumption of each UAV during delivery while adhering to the deadline constraints of the delivery orders.

An Integrated Utility Function (IUF_1) that considers the deadline constraint of the delivery service request when evaluating the delivery service selection plan was designed, as shown in Objective (13). In this paper, the optimization goal of the delivery service selection problem is to minimize the integrated utility value of the fitness ($FIUV_1$) [9]. Therefore, based on the above model, the delivery service selection optimization problem can be formulated as follows:

$$\min FIUV_1 = \begin{cases} DE_{\text{sum}}; & DT_{\text{sum}} \leq DL_{\text{sum}} \\ DE_{\text{sum}} + \frac{DT_{\text{sum}}}{DL_{\text{sum}}} \times PE; & DT_{\text{sum}} > DL_{\text{sum}} \end{cases} \quad (13)$$

where DE_{sum} and DT_{sum} are the total delivery energy consumption and time for delivery order set $\mathbf{O} = \{O_1, O_2, \dots, O_{N_O}\}$ respectively, DL_{sum} is the delivery order's deadline, and PE is the penalty constant value.

Definition 8: Computational service optimization problem

In this paper, the optimization goal of the computational service optimization problem is to minimize the integrated utility value of the fitness ($FIUV_2$). The goal is to optimize the service response time of the computational service. The service response time must meet or exceed the required deadlines to ensure the availability and reliability of subsequent requests. An integrated utility function (IUF_2) that considers the total service response time when evaluating the computational service selection plan was designed, as shown in (14). The optimization goal of the computational service selection problem is to minimize the

IUF_2 value of the fitness ($FIUV_2$). Hence, relying on the aforementioned model, the optimization problem for computational service selection can be formulated as follows:

$$\min FIUV_2 = \min \left\{ \sum_{i=1}^{N_O} \sum_{j=1}^{N_U} CT_{i,j} \right\} \quad \text{s.t. } CT_{i,j} \leq CR_j \cdot DL^C \quad (14)$$

The total computational service response time is the sum of each computational service response time $CT_{i,j}$ for computational service $CS_{i,j}$. This objective is designed to ensure that the response time remains below the computational service request CR_j 's deadline DL^C , thus avoiding task execution failures that could affect subsequent tasks.

IV. PROPOSED SOLUTION

In this section, a two-stage holistic and hybrid service selection (H^2S^2) strategy for MEC-based UAV last-mile delivery systems is proposed as an integral approach for optimizing multi-UAV service selection problems in MEC-based UAV last-mile delivery systems. First, we present an overview of the H^2S^2 strategy. Then, the details of two service selection algorithms for solving the delivery and computational service selection optimization problems are discussed.

A. Overview of the H^2S^2 Strategy

Real-world MEC-based UAV last-mile delivery systems have two main stages: the static stage and the dynamic stage. First, when the delivery system receives the user's delivery order, the system needs to analyse the characteristics of the user's order and generate various service selection plans. Second, when the UAV begins to deliver the package, the system needs to provide a dynamic service selection plan in case there are service availability or conflict issues. Therefore, the H^2S^2 strategy is also divided into two stages. In the static stage, before the UAV takes off, the cloud servers generate static delivery service and computational service selection plans according to the characteristics of the service requests. After the UAV takes off, the dynamic stage resolves service availability or conflict issues with service re-selection algorithms. It is obvious that the delivery and computational service selection problem focused in this paper is a typical NP-hard problem. Since there is no guarantee to find the global optimal solution for a NP-hard problem, the practical target of the H^2S^2 strategy proposed in this paper is to find a near-optimal solution which is the best solution found by the optimization algorithm within a limited amount of running time.

Fig. 5 illustrates the schematic diagram of the H^2S^2 strategy in MEC-based UAV last-mile delivery systems. The strategy is categorized into two layers according to the two types of service requests. Each layer has two optimization stages: the static and dynamic stage. In the static phase, the UAV delivery service is not started. There is no real-time requirement for the delivery system at this time. Therefore, we use the particle swarm optimization-based delivery and computational service selection algorithm that is capable of time consumption and close to the optimal solution. To mitigate the risk of the PSO algorithm converging to local optima, we employed diverse initialization, velocity clamping, adaptive inertia weight adjustment and periodic re-initialization strategies. In this context,

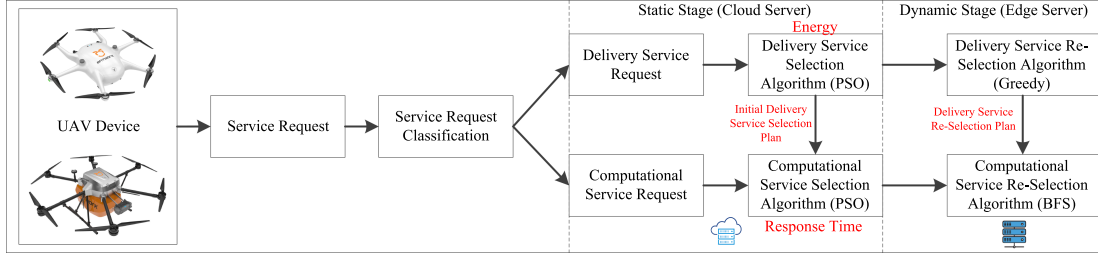
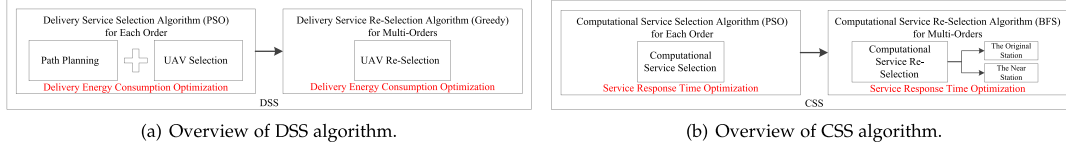
Fig. 5. Flowchart of the H^2S^2 strategy.

Fig. 6. Overview of the DSS and CSS algorithm.

due to the collaborative optimization of delivery and computational services facilitated by the proposed H^2S^2 strategy, the generation of computational service selection schemes relies on delivery service plans. Consequently, the initial delivery plans need to be provided as algorithm input to the computational service selection algorithm. In the dynamic phase, we use greedy and BFS-based delivery and computational service re-selection algorithms with low time complexity to solve the service re-selection problem due to the real-time requirements in the UAV delivery process. Similar to the static stage, the computational service selection algorithm in the dynamic stage also requires delivery service selection plans as algorithm input.

B. Delivery Service Selection (DSS) Algorithm

An overview of the delivery service selection (DSS) algorithm is shown in Fig. 6(a). This algorithm can be divided into two parts. The first is a PSO-based delivery service selection algorithm that considers the UAV flight path planning problem and the allocation problem (contain UAV selection and delivery relay service selection) between the UAV and the package. The second is a greedy-based delivery service re-selection algorithm that resolves delivery service failures or conflicts. The optimization objective of the DSS algorithm is to minimize the UAV's delivery energy consumption under the delivery order deadline constraint. All algorithms' time complexity analyses are detailed in Appendix C, available online.²

Algorithm 1 demonstrates the PSO-based delivery service selection algorithm. This algorithm has five phases. Phase I is the UAV flight path planning. In this phase, for each delivery order, the UAV's best flight path is generated by the Dijkstra shortest path algorithm according to the start and destination points (Lines 1–5). Phase II initializes the UAV's delivery service selection plan based on the best flight path (Lines 6–9). After the initial delivery service selection, the plan is generated for all delivery orders. Phase III determines the best initial delivery service selection plan by calculating the value of $FIUV_1$ (Lines 10–11). Then, Phase IV enters the algorithm iteration process (Lines 12–20). This phase has four parts: update the delivery service selection plan according to the search speed (Line 14),

Algorithm 1: PSO-Based Delivery Service Selection Algorithm.

Input: the delivery order set O , the service request set DR , algorithm iterations $ITER$, the flight path FR , the initial delivery service plan set $init_DS$, population size P ;
Output: DS_{best} , $FIUV_{best}$; $\triangleright$ the best delivery service selection plan; the integrated utility value of DS_{best} ;

```

1: procedure
2:   for  $i = 1$  to  $N_o$  do
3:      $S\_path = \text{Function\_Dijkstra}(FR, SP_i)$ 
4:      $BF_i = \text{Find}(S\_path, DP_i) \triangleright$  find the shortest path;
5:   end for
6:   for  $i = 1$  to  $N_o$  do
7:     initialize delivery service plan set  $init\_DS_i$ , search speed  $v_i$  according to the  $BF_i$ 
8:     calculate  $MR$ ,  $DT_{sum}$ ,  $DE_{sum}$ ,  $FIUV_1$  of delivery service plan  $init\_DS_i$  (Eqs. (10)–(12))
9:   end for
10:  choose the global best delivery service plan from  $init\_DS_i$ , then store its  $FIUV_1$  into  $FIUV_{best}$  and store the best delivery service plan into  $DS_{best}$ 
11:  select the delivery service plan from  $init\_DS_i$  by the roulette selection strategy and store them into  $init\_DS_i$ 
12:  while  $i < ITER$  do
13:    for  $j = 1$  to  $P$  do
14:      update delivery service selection plan  $init\_DS_j$  by  $v_j$ 
15:      calculate  $DT_{sum}$ ,  $DE_{sum}$ ,  $FIUV_1$  of delivery service plan  $init\_DS_j$  (Eqs. (10)–(12))
16:    end for
17:    choose the global best delivery service plan from  $init\_DS_j$ , then store its  $FIUV_1$  into  $FIUV_{best}$  and store the best delivery service plan into  $DS_{best}$ 
18:    update inertia weight  $w$ 
19:    update search speed  $v_i$ 
20:  end while
21:  return  $DS_{best}$ ,  $FIUV_{best}$ 
22: end procedure

```

Algorithm 2: Greedy-Based Delivery Service Re-Selection Algorithm.

Input: the best delivery service selection plan DS_{best} , current time s ;
Output: SR_{best} ; $\triangleright$ the best delivery service re-selection plan

```

1: procedure
2:   for each time slot  $t$  do
3:      $s = s + t$ 
4:     for  $i = 1$  to  $N_o$  do
5:        $(j, k) = Get\_IDDS_i$   $\triangleright$  get the current in use delivery service
6:        $CD = Get\_Destination(DS_{j,k}, FR_{j,k})$   $\triangleright$  get the destination point of current delivery task
7:        $CU = Position(U_j, FR_{j,k})$   $\triangleright$  get the current position of UAV
8:       if  $Distance(CD, CU) < SD$  then
9:         if  $(Check\_Avail(DS_{j,k+1}) == 0)$  then  $\triangleright$  check the current status of  $DS_{j,k+1}$ 
10:           $AS = Sort\_Avail\_Service(DS^D)$   $\triangleright$  sorts the available relay delivery services
11:          for  $m = 1$  to  $N_{AS}$  do
12:            if  $ML_{i,m} > W_i$  then  $\triangleright$  check the maximum payload of  $DS_{i,m}$  does not exceed the weight of the parcel  $O_i$ 
13:               $SR_{best} = DS_{i,m}^D$   $\triangleright$  select the best relay delivery service
14:            end if
15:          end for
16:        end if
17:      else
18:        continue
19:      end if
20:      if  $Order\_finish(O) == 1$  then
21:        break
22:      end if
23:    end for
24:  end for
25:  return  $SR_{best}$ 
26: end procedure

```

calculate the value of $FIUV_1$ (Line 15), determine the global best computational service selection plan (Line 17) and update the algorithm parameters (Lines 18–19). In Phase V, the algorithm returns the best delivery service selection plan DS_{best} and the value of $FIUV_{best}$ (Line 21).

Algorithm 2 aims to resolve delivery service availability issues and conflicts by using a greedy strategy. This algorithm can be divided into five main phases. First, Phase I uses DS_{best} (generated by Algorithm 1) to start the UAV's delivery process (Lines 1–5). Then, Phase II obtains the status of the UAV at the current time, including the current delivery service selection plan and the UAV's location (Lines 6–7). When the delivery UAV approaches the relay point, Phase III judges the availability of the next delivery service selection plan (Lines 8–9). If the next delivery service selection plan is unavailable, Phase IV sorts the available services in the relay delivery station and chooses the best delivery service selection plan with the greedy strategy (Lines 10–22). Finally, Phase V returns the best delivery service re-selection plan SR_{best} (Line 25).

Algorithm 3: PSO-Based Computational Service Selection Algorithm.

Input: O , The computational service request set CR , $ITER$, FR , the initial computational service plan set $init_CS$, P , DS_{best} ;
Output: CS_{best} , $FIUV_{best}$; $\triangleright$ the best computational service selection plan; the integrated utility value of CS_{best}

```

1: procedure
2:   for  $i = 1$  to  $N_o$  do
3:     initialize computational service plan set  $init\_CS_i$  according to  $DS_{best}$ , search speed  $v_i$  according to the  $CR_i$ 
4:     calculate  $CT_i$ ,  $FIUV_2$  of computational service plan  $init\_CS_i$  (Eqs. (6),(13)–(14))
5:   end for
6:   choose the global best computational service plan from  $init\_CS_i$ , then store its  $FIUV_2$  into  $FIUV_{best}$  and store the best delivery service plan into  $CS_{best}$ 
7:   select the computational service plan from  $init\_CS_i$  by the roulette selection strategy and store them into  $init\_CS_i$ 
8:   while  $i < ITER$  do
9:     for  $j = 1$  to  $P$  do
10:      update computational service selection plan  $init\_CS_j$  by  $v_j$ 
11:      calculate  $CP_j$ ,  $TT_j$ ,  $FIUV_2$  of delivery service plan  $init\_CS_j$  (Eqs. (7)–(8),(14))
12:    end for
13:    choose the global best delivery service plan from  $init\_CS_j$ , then store its  $FIUV_2$  into  $FIUV_{best}$  and store the best delivery service plan into  $CS_{best}$ 
14:    update inertia weight
15:    update search speed  $v_i$ 
16:  end while
17:  return  $CS_{best}$ ,  $FIUV_{best}$ 
18: end procedure

```

C. Computational Service Selection (CSS) Algorithm

Fig. 6(b) provides an overview of the Computational Service Selection (CSS) algorithm. The CSS algorithm consists of two parts. The first is a PSO-based computational service selection algorithm for each delivery order, which focuses on the UAV's computational service selection problem. The second is a BFS-based computational service re-selection algorithm for multiple delivery orders, which focuses on computational service availability issues and conflicts. The optimization goal of the CSS algorithm is to minimize the computational service response time of the UAV.

Algorithm 3 demonstrates the PSO-based computational service selection algorithm. This algorithm has three phases. Phase I initializes the UAV's computational service selection plan and determines the best initial computational service selection plan by calculating the value of $FIUV_2$ (Lines 1–7). Then, Phase II enters the algorithm iteration process (Lines 8–16). This phase has four parts: update the computational service selection plan according to the search speed (Line 10), calculate the value of $FIUV_2$ (Lines 11–12), determine the global best computational service selection plan (Line 13) and update the

Algorithm 4: BFS-Based Computational Service Re-Selection Algorithm.

Input: CS_{best} , s , SR_{best} ;
Output: RC_{best} ; $\triangleright$ the best computational service re-selection plan

```

1: procedure
2:   for each time slot  $t$  do
3:      $s = s + t$ 
4:     for  $i = 1$  to  $N_o$  do  $k = 0$ 
5:       initialize visited[] by 0
6:        $CU = \text{Position}(SR_{best}, U_i, s)$ 
7:        $(j, k) = \text{Get}_{ID}(CS_i)$   $\triangleright$  get the current in use computational service
8:       if  $\text{Check\_Availability}(CS_{j,k} == 0)$  then  $\triangleright$  check the current status of  $CS_{j,k}$ 
9:          $q = \text{NewQueue}$ 
10:         $q.\text{EnQueue}(ST_j)$ 
11:        while  $q \neq \emptyset$  do
12:           $x = q.\text{GetHead}()$ 
13:          if  $ST_x$  has not been visited then
14:             $BQ_k = ST_x$ 
15:             $\text{visited}[x] = \text{true}$ 
16:            calculate  $BQ_k$ ,  $FIUV_2$  of computational service re-selection plan  $RC_{x,k}$ 
17:             $k = k + 1$ 
18:            for each  $ST_y$  connected to  $ST_x$  do
19:              if  $\text{visited}[y] == \text{false}$  then
20:                 $q.\text{enqueue}(ST_y)$ 
21:              end if
22:            end for
23:          end if
24:        end while
25:         $\triangleright$  Update  $RC_{best}$  based on the selected service
26:         $RC_{best} \leftarrow \text{ReSelection\_Plan}(RC_{x,k})$ 
27:      end if
28:    end for
29:  return  $RC_{best}$ 
30: end procedure

```

algorithm parameters (Lines 14–15). In Phase III, the algorithm returns the best computational service selection plan CS_{best} and value of $FIUV_{best}$ (Line 17).

Algorithm 4 describes the BFS-based computational service re-selection algorithm, which focuses on computational service availability issues and conflicts. This algorithm has four phases. Phase I: The algorithm initializes the UAV's computational service process using CS_{best} generated by Algorithm 3. This involves iterating through each time slot t and each UAV i , setting up initial conditions and variables (Lines 1–5). Phase II: The algorithm obtains the UAV's flight status in the current time slot, which includes the UAV's position and computational service plan. This phase involves determining the current computational service in use (Lines 6–7). Phase III: The algorithm verifies the availability of the current computational service. If the current service is unavailable, the algorithm initializes a BFS process to explore alternative services. It calculates BQ_k and $FIUV_2$ for each service re-selection plan $RC_{x,k}$. Using the BFS strategy, it iteratively explores all connected services to find the best option and updates RC_{best} based on the selected service (Lines 8–26).

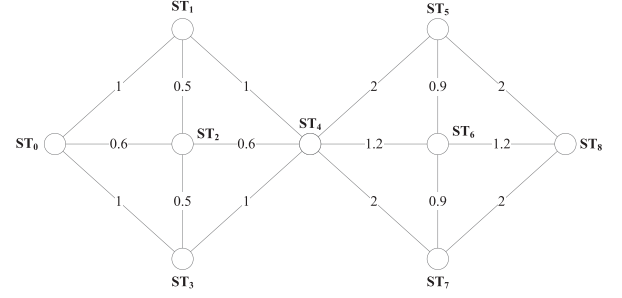


Fig. 7. The simulation environment of the UAV delivery system.

TABLE I
PARAMETER SETTINGS OF DIFFERENT SERVICE RESOURCES

Service Types	Delivery Service			Computational Service	
	Payload (kg)	Flight Power (W)	Flight Speed (km/h)	Cloud Server (MIPS)	Edge Server (MIPS)
Type 1	5.5	1000	55		1000
Type 2	0.83	202	43	2500	1300
Type 3	1.13	356	62		1600
Type 4	1	444	77		-

Phase IV: The algorithm returns the best computational service re-selection plan RC_{best} (Line 29).

V. EVALUATION

Since the H^2S^2 strategy has two phases, we evaluate not only the performance of the overall H^2S^2 strategy but also the performance of the individual phases by using a thorough ablation study. Additionally, we assess the impact of algorithm parameters such as the penalty constant PE and the number of algorithm iterations on the H^2S^2 strategy (Appendix D, available online²). To highlight the significance of mobile edge computing for computational service requests, we present the distribution of computational service requests under the H^2S^2 strategy (Appendix E, available online²).

A. Simulation Environment

In the simulation, we abstracted the UAV delivery simulation environment based on a real-world UAV automatic delivery network (ADNET project in Appendix A.1, available online²). Fig. 7 shows that there are nine delivery stations and several flight paths. Each delivery station has different types of delivery and computational service resources. The data transmission rate is determined based on the average distance from the edge server base station, divided into three levels: 2048 KB per second, 1024 KB per second, and 512 KB per second [27]. The UAV delivery scenario considered in our study supports the scheduling of multiple UAVs simultaneously. The details of the service resources are shown in Table I [21]. We design six delivery orders based on these parameters. The delivery start and destination points, cargo weight demands and deadline constraints are shown in Table II [21]. The ranges of the computational service input and output data are (15 MB, 90 MB) and (5 MB, 30 MB), respectively. Each edge server can cover

TABLE II
PARAMETER SETTINGS OF DIFFERENT DELIVERY ORDERS

Order ID	Cargo Weight (kg)	Start Point	Destination Point	Deadline (s)
1	0.8	ST_1	ST_2	39.0
2	0.8	ST_2	ST_0	47.0
3	0.9	ST_1	ST_3	78.0
4	0.9	ST_0	ST_4	94.0
5	1	ST_0	ST_6	189.0
6	1	ST_2	ST_8	236.0

 TABLE III
PARAMETER SETTINGS OF THE PSO ALGORITHM

Parameter Name	Setting
Number of particles in the swarm	30
C1	2
C2	2
Inertia weight factor	0.8-1.2

requests for computing services within a 1km radius [9]. In our current scenario, we have ensured that each UAV during the delivery process is within the signal range of at least one edge server. To simplify the service selection problem considered in the paper, the additional overhead of service migration is not considered. The baseline computational service processing time is generated by (500MI, 9800MI) [9]. Furthermore, we create the simulation environment utilizing the open-source EXPRESS project.³ EXPRESS is recognized for its energy-efficient and secure framework, particularly designed for MEC and blockchain-based smart systems. [25].

The simulation experiments are implemented by JDK 1.8.⁴ All Presently, there exist two distinct categories of methods for addressing service management issues in cloud or MEC environments: heuristic-based methods and machine learning-based methods [19], [28], [29], [30], [31]. Therefore, we compare the performance of the H^2S^2 strategy with four different heuristic-based and machine learning-based approaches. Alternative approaches include: (1) a genetic algorithm (GA)-based energy-efficient service selection strategy (ESG) [21]; (2) a PSO-based energy-efficient service selection strategy (ESP) [19], [21]; (3) a crowding distance (CD)-based energy-efficient service selection strategy (ESC) [1]; (4) an all-in-cloud strategy (computational service selection only). The acceleration of gravity g is 10 m/s^2 . The fluid density of air ρ is 1.204 kg/m^3 . The area of the rotating blade ζ is 0.2m^2 [22]. The parameter setting for the PSO and GA strategies are presented in Tables III and IV. [21].

B. Performance Evaluation of the Delivery Service Selection Strategy

To validate the effectiveness and efficiency of the proposed strategy in identifying the optimal delivery service selection plan, the H^2S^2 strategy was compared with three other methods (ESG, ESP, ESC) in terms of four indicators: the UAV's delivery

 TABLE IV
PARAMETER SETTINGS OF THE GA ALGORITHM

Parameter Name	Setting
Number of chromosomes	50
Crossover rate	0.1
Mutation rate	0.02
Number of iterations	100

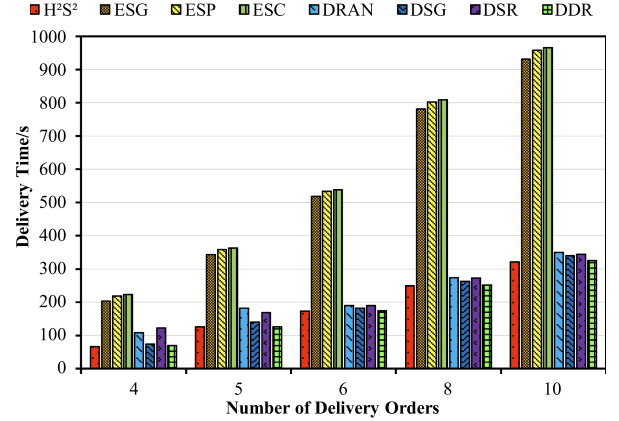


Fig. 8. Experimental results of the UAV's delivery time.

energy consumption for effectiveness, the UAV's delivery time for efficiency, the delivery efficiency and the run time of the different methods. Ablation studies were used to demonstrate the relative influence of the two subalgorithms in the DSS algorithm, and four variants of the DSS algorithm were generated: an all-stage random algorithm (DRAN), a static greedy subalgorithm (DSG), a static random subalgorithm (DSR), a dynamic random subalgorithm (DDR).

1) *UAV's Delivery Time*: Fig. 8 depicts the experimental results of the UAV's delivery time. The proposed H^2S^2 strategy has the lowest UAV delivery time. For the three typical algorithms, when the number of delivery orders is 6, the UAV delivery time with the H^2S^2 strategy is 66.64%, 69.57% and 67.86% lower than that with the ESG, ESP and ESC strategies, respectively. When the number of delivery orders is 10, the UAV delivery time with the H^2S^2 strategy is 65.48%, 66.48% and 66.75% lower than that with the ESG, ESP and ESC strategies, respectively. The reason for this result is that the H^2S^2 strategy significantly reduces the service waiting time in the UAV relay delivery process by resolving conflicts in the delivery service. The ablation study shows that the results of the H^2S^2 strategy were lower than the results of all four DSS variants. For example, when the number of delivery orders is 6, the UAV delivery time with the H^2S^2 strategy is 8.92%, 5.11%, 8.92%, and 6.63% lower than the delivery times with the DRAN, DSG, DSR, and DDR algorithms. In summary, the efficiency of the two-stage H^2S^2 strategy is verified.

2) *UAV's Delivery Energy Consumption*: The UAV's delivery energy consumption with the seven methods is shown in Fig. 9. The UAV's delivery energy consumption with the H^2S^2 strategy is always lower than the delivery energy consumption with the three typical algorithms when the number of delivery orders is larger than 2. For example, when the number of delivery orders is 6, the UAV's delivery energy consumption with the H^2S^2 strategy is 12.26%, 11.22% and 22.78% lower than that

³EXPRESS 2.0 platform: <https://github.com/ISEC-AHU/EXPRESS2.0>

⁴Experiment dataset and source code: <https://github.com/ISEC-AHU/HHSS>

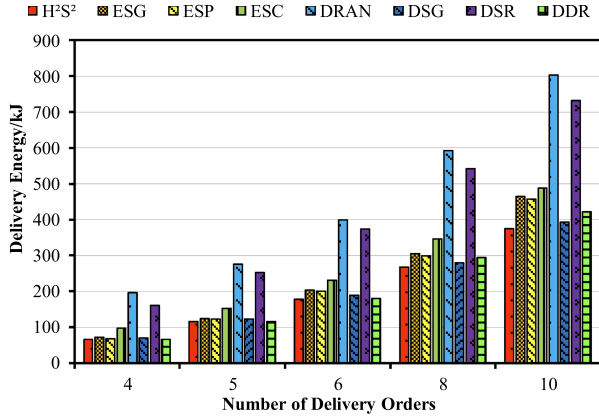


Fig. 9. Experimental results of the UAV's delivery energy consumption under the different number of delivery orders.

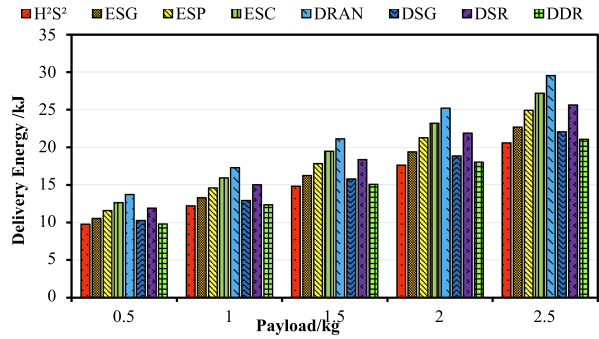


Fig. 10. Experimental results of the UAV's delivery energy consumption under the different payload.

with the ESG, ESP and ESC strategies, respectively. When the number of delivery orders is 10, the UAV's delivery energy consumption with the H^2S^2 strategy is 19.34%, 18.21% and 23.51% lower than that with the ESG, ESP and ESC strategies, respectively. This result indicates that the H^2S^2 strategy usually finds the best delivery service selection plan for different delivery orders. Moreover, the H^2S^2 strategy can quickly switch to the candidate solution if service conflicts occur. The ablation study shows that the result of the H^2S^2 strategy is lower than the results of all four DSS variants. For example, when the number of delivery orders is 6, the UAV's delivery energy consumption with the H^2S^2 strategy is 55.27%, 5.56%, 52.20% and 21.84% lower than the delivery energy consumption with the DRAN, DSG, DSR and DDR algorithms. In conclusion, the H^2S^2 strategy is more effective than the other three strategies in terms of the energy consumption of the UAV.

To further evaluate the impact of the payload factor on the energy consumption optimization of the H^2S^2 strategy, we conducted experiments on the UAV's delivery energy consumption under different payloads. Fig. 10 illustrates that the H^2S^2 strategy maintains the lowest energy consumption for UAV delivery under different payloads. For example, when the payload is 1.5kg, the UAV's delivery energy consumption with the H^2S^2 strategy is 8.62%, 16.80% and 23.73% lower than that with the ESG, ESP and ESC strategies, respectively. When the payload is increased to 2.5kg, the H^2S^2 strategy still maintains the lowest UAV's delivery energy consumption which is lower than the ESG, ESP and ESC strategies by 9.23%, 17.35% and 24.24%, respectively, showing a stable performance. Furthermore, the

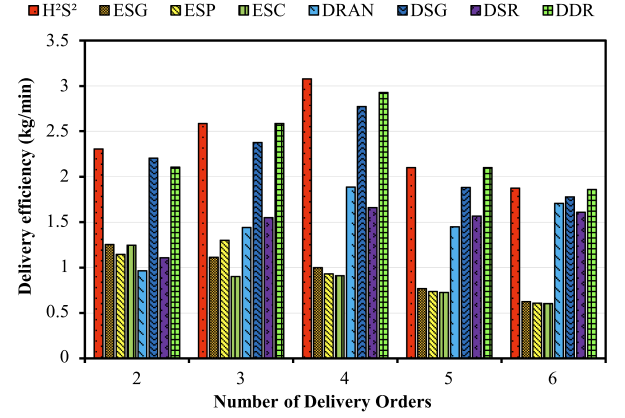


Fig. 11. Experimental results of the UAV's delivery efficiency.

ablation study shows that the result of the H^2S^2 strategy is better than that of all four DSS variants. For example, when the payload is 1.5kg, the UAV's delivery energy consumption with the H^2S^2 strategy is 29.79%, 6.08%, 19.14% and 1.68% lower than the delivery energy consumption with the DRAN, DSG, DSR and DDR algorithms, respectively.

3) *Delivery Efficiency*: Delivery efficiency is one of the most important indicators for a delivery system. In this paper, the delivery efficiency of different methods was investigated by considering the maximum weight delivered within the same time period. Thus, the higher the weight of the cargo delivered at the same time, the better the corresponding service selection strategy. Fig. 11 shows that the maximum delivery weight of the proposed H^2S^2 strategy is higher than the maximum delivery weights of the three typical algorithms. For instance, when the quantity of delivery orders is 2, the maximum delivery weight of the H^2S^2 strategy is 83.59%, 100.87% and 84.76% higher than that of the ESG, ESP and ESC strategies, respectively. When the number of delivery orders is 6, the maximum delivery weight of the H^2S^2 strategy is nearly three times of the ESG, ESP and ESC strategies. The H^2S^2 strategy reduces unnecessary energy and time consumption in the delivery and computational service selection processes, thus increasing the efficiency of how the UAVs transport cargo. The ablation study shows that the result of the H^2S^2 strategy is higher than the results of all four DSS variants. For example, when the number of delivery orders is 6, the UAV's delivery energy consumption with the H^2S^2 strategy is 9.78%, 5.40%, 16.60% and 7.18% lower than those with the DRAN, DSG, DSR and DDR algorithms.

4) *Run Time of the Different Methods*: Because MEC-based UAV last-mile delivery systems must operate in real time, the running efficiency of the proposed strategy is extremely important. Fig. 12 illustrates the execution time of different strategies. This experiment involved running the H^2S^2 , ESG, ESP, and ESC strategies 100 times and computing the average execution time. Notably, the H^2S^2 strategy demonstrates significantly superior performance compared to the other three strategies. As the number of delivery orders increases, the execution times of the H^2S^2 , DSG, DSR and DDR strategies increases slowly. In contrast, the execution times of other strategies increase rapidly as the number of delivery orders increases. This result occurs because the execution time of the H^2S^2 strategy is heavily dependent on the number of delivery and computational service resources and less dependent on the number of delivery

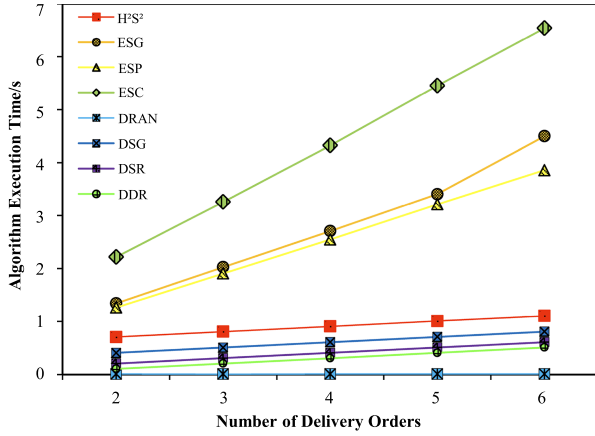


Fig. 12. Experimental results pertaining to the execution time of various strategies.

orders. Therefore, once the environment of the MEC-based UAV last-mile delivery systems is confirmed, the execution time of the H^2S^2 strategy stabilizes. The ablation study shows that the execution time of the H^2S^2 strategy is larger than the execution times of all four DSS variants. However, there is a large gap between the H^2S^2 strategy and the DSS variants in terms of the delivery time, UAV energy consumption, and delivery efficiency experimental results. In summary, the efficiency of the two-stage H^2S^2 strategy is verified.

C. Performance Evaluation of the Computational Service Selection Strategy

To verify the effectiveness and efficiency of the proposed strategy in determining the optimal computational service selection plan. The H^2S^2 strategy was compared with nine other methods in terms of two indicators: the computational service response time and the service deadline miss rate. Ablation studies were used to demonstrate the relative influence of the two subalgorithms in the CSS algorithm. To this end, five variants of the CSS algorithm were generated: an all-stage random algorithm (CRAN), a static greedy algorithm (CSG), a static random algorithm (CSR), a dynamic random algorithm (CDR), and a static-only algorithm (CSO).

1) *Computational Service Response*: For the nine typical algorithms, an experiment was conducted in which the number of computational service requests was changed from 200 to 600, and the H^2S^2 , all-in-cloud, ESG, ESP and ESC strategies were run 100 times, yielding 100 results. Then, the average computational service response time was calculated, as shown in Fig. 13. The figure shows that the H^2S^2 strategy has the shortest computational service response time. In addition, the gap between the H^2S^2 strategy and the other strategies gradually increases as the number of computational service requests increases. As an example, consider the case with 200 computational service requests; the computational service response time of the H^2S^2 strategy is 8.06%, 9.40%, 23.58% and 27.77% lower than that of the ESG, ESP, ESC and all-in-cloud strategies, respectively. When the number of computational service requests is increased to 600, the computational service response time of the H^2S^2 strategy is 8.06%, 9.40%, 25.79% and 29.02% lower than that of the ESG, ESP, ESC and all-in-cloud strategies, respectively. The ablation study shows that the computational service response

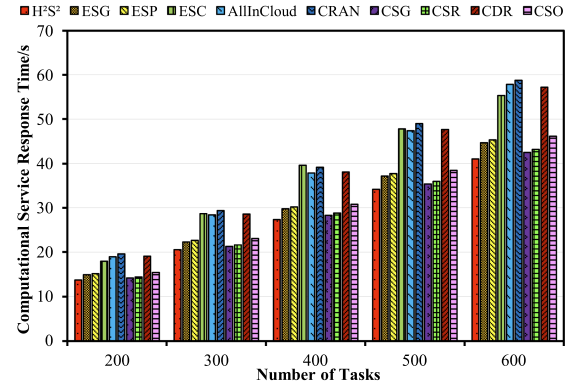


Fig. 13. Experimental results of the computational service response time.

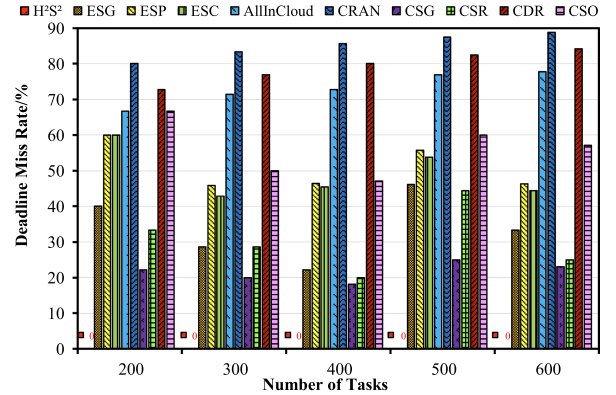


Fig. 14. Experimental results of the deadline miss rate.

time of the H^2S^2 strategy is lower than the computational service response times of all five CSS variants. For example, when the number of tasks is 200, the UAV's computational service response time with the H^2S^2 strategy is 30.13%, 11.05%, 3.39%, 5.00% and 28.23% lower than that of the CRAN, CSO, CSG, CSR and CDR algorithms, respectively. Therefore, the H^2S^2 strategy has better efficiency in terms of determining the optimal computational service selection plan in the MEC environment.

2) *Deadline Miss Rate*: The deadline miss rate of the computational service requests is shown in Fig. 14. Despite the increasing number of computational service requests, the deadline miss rate of the H^2S^2 strategy remains at nearly 0 for all cases. All other strategies have much larger deadline miss rate. This result indicates that the proposed strategy can effectively prevent computational service unavailability through service selection or re-selection algorithms for the UAV.

VI. RELATED WORK

The research status of two service selection problems are presented in this section. The first is the general computation service selection problem in cloud or MEC environments. The second is the delivery service selection optimization problem in the MEC-based UAV last-mile delivery scenario.

A. Computation Service Selection

The computational service selection problem has emerged as a prominent research focus in the field of service computing [32],

TABLE V
RELATED WORKS AND THEIR PROPERTIES

Reference	Objective	Computation Service	Delivery Service	Static Service	Dynamic Service	Methodology
[30]	Cost	✓	×	✓	×	Machine learning algorithm
[29]	Cost	✓	×	✓	×	Heuristic algorithm
[31], [36]	Energy consumption	✓	×	✓	×	Evolution algorithm
[37]	QoS	✓	×	×	✓	Collaborative filtering based method
[38]	Delivery time	×	✓	✓	×	Modified A* algorithm
[8]	Delivery efficiency	×	✓	×	✓	Machine learning algorithm
This paper	Energy consumption, Response time	✓	✓	✓	✓	H^2S^2

[33]. These research endeavors can be categorized into two main groups based on the application environment type: cloud computing environments and Multi-Access Edge Computing (MEC) environments.

In the context of cloud computing environments, a considerable body of research has concentrated on the optimization problem of service selection. Most studies consider three pivotal factors: the user's QoS, the mobility of end devices, and the network environment [2], [34], [35]. For instance, Yu et al. [30] introduces FaaS Deliver, an adaptive and efficient function delivery engine for optimizing cost in a heterogeneous computing continuum (HCC) comprising. FaaS Deliver identifies cost-efficient function delivery policies (FDPs) for FaaS platform selection and resource allocation. Real-world evaluations show that FaaS Deliver reduces function execution costs by 38% to 78% compared to state-of-the-art methods. Sun et al. [29] introduces the Enhanced Task Type First Algorithm (ET2FA) for cost-efficient, deadline-constrained workflow scheduling in cloud computing. ET2FA minimizes total cost and idle rates using task type scheduling, delay optimization, and instance hibernation. Simulations show ET2FA outperforms state-of-the-art algorithms.

In the MEC environment, several research endeavors have focused on optimization problems related to service selection. For example, Peng et al. [36] investigated the impact of safety factors on service selection and task offloading plans during UAV flight, proposing a multi-objective evolutionary algorithm. Experimental results demonstrated that the proposed algorithm can optimize energy consumption in service selection and task offloading plans while ensuring the safety of UAV flight. Yin et al. [31] proposes an Efficient Convergent Firefly Algorithm (ECFA) for scheduling security-critical tasks in cloud-edge computing. ECFA uses a probability-based mapping operator and low-complexity position update strategy, ensuring efficient solution exploration. Theoretical analysis and evaluations demonstrate ECFA's superiority and robustness against boundary traps compared to other metaheuristics. Wang et al. [37] introduced a collaborative filtering-based service recommendation method in the MEC environment, which simultaneously considered user mobility, service availability, and user Quality of Service (QoS) requirements. Experimental results affirmed that the proposed method enhances the accuracy of the service selection plan in the MEC environment.

Upon analyzing the existing research on computing service selection problems in cloud or MEC environments, it is evident

that the majority of studies concentrate on optimizing the computing service selection problem. However, these efforts often overlook the diverse range of services in MEC environments. To tackle this gap, we propose a holistic solution for addressing the diverse service selection problem in MEC-based UAV last-mile delivery systems. This solution takes into account both computational service and delivery service selection problems.

B. UAV Service Selection

Numerous research works have addressed the UAV delivery service selection problem [8], [21], [38]. For instance, Alkouz et al. [38] introduced a swarm-based drone-as-a-service framework for delivery systems. Within this framework, two service selection algorithms were proposed, taking into account various behaviors of drone swarms. Experimental results demonstrated that the proposed algorithms enhanced the delivery efficiency of the drone delivery system. Chu et al. [21] presented a two-stage energy-efficient service provision strategy based on the genetic algorithm (ESP-GA) to address the delivery service and computational service selection problems in Multi-Access Edge Computing (MEC)-based UAV last-mile delivery systems. Experimental results showcased the improved performance of the proposed strategy in terms of UAV energy consumption and delivery time. Addressing practical constraints in the UAV delivery process, existing studies have also proposed solutions. Hamdi et al. [8] considered the drone delivery service selection problem with uncertainty factors, aiming to maximize the efficiency and effectiveness of the delivery system. A comprehensive solution, featuring an uncertainty-aware delivery service selection architecture and several service selection algorithms, was proposed. The suggested solution facilitates efficient and effective delivery services within the delivery system.

The literature review presented in Table V highlights that existing research on the UAV service selection problem has predominantly focused on either delivery service selection or computational service selection, with limited attention to both static and dynamic service selection. This fragmented approach fails to address the comprehensive, multi-stage complexity of the problem, potentially leading to prolonged response time and increased energy consumption in UAV delivery service schemes.

To address this gap, this paper proposes a mobility-aware hybrid service selection (H^2S^2) strategy for MEC-based UAV last-mile delivery systems. This strategy adopts a holistic perspective by considering both static delivery service selection and

static computing service selection, a hybrid perspective by incorporating dynamic delivery service re-selection and dynamic computing service re-selection. The primary objective of the H^2S^2 strategy is to optimize UAV energy consumption while ensuring efficient service response times.

VII. CONCLUSION AND FUTURE WORK

This paper introduces a comprehensive and hybrid service selection strategy (H^2S^2) tailored for multi-access edge computing (MEC) in unmanned aerial vehicle (UAV) last-mile delivery systems. The H^2S^2 strategy is designed to mitigate the UAV's energy consumption during delivery and enhance computational service response time, taking into account various influencing factors in the given scenario that impact UAV service plan selection. Experimental evaluations were conducted on the ADNET project, a real-world UAV delivery system. The H^2S^2 strategy was thoroughly assessed alongside other representative strategies, focusing on optimization objectives related to delivery and computational service selection. The results of the experiments demonstrate that the H^2S^2 strategy significantly enhances the efficiency and effectiveness of the UAV delivery system by reducing both energy consumption and service response times.

The service selection problem in multi-access edge computing-based UAV last-mile delivery systems remains a key challenge. This paper focused on optimizing the delivery energy consumption and the computational service response time. However, the complex relationships between different service requests were not considered. Therefore, future research work should consider the complex relationships between service requests by combining different technology workflows.

Additionally, the issue of UAV path conflicts when scheduling multiple UAVs simultaneously in close proximity, such as within a one-mile radius, was not considered in this study. Future work will investigate how to effectively manage and resolve potential path conflicts to enhance the robustness and efficiency of the delivery algorithm. This will involve developing strategies for dynamic path planning and conflict resolution in dense urban environments.

ACKNOWLEDGMENT

Special thanks to the Antwork Robotics Company, Ltd.

REFERENCES

- [1] H. Guo and J. Liu, "UAV-enhanced intelligent offloading for Internet of Things at the edge," *IEEE Trans. Ind. Informat.*, vol. 16, no. 4, pp. 2737–2746, Apr. 2020.
- [2] R. Mahmud, K. Ramamohanarao, and R. Buyya, "Application management in fog computing environments: A taxonomy, review and future directions," *ACM Comput. Surv.*, vol. 53, no. 4, pp. 1–43, 2020.
- [3] P. Taylor, "Number of connected devices worldwide in 2014 and 2028, by device," [EB/OL]. Accessed: Jun. 4, 2024. [Online]. Available: <https://www.statista.com/statistics/512650/worldwide-connected-devices-amount/>
- [4] L. Wang, Q. He, D. Gao, J. Wan, and Y. Zhang, "Temporal-perturbation aware reliability sensitivity measurement for adaptive cloud service selection," *IEEE Trans. Services Comput.*, vol. 15, no. 4, pp. 2301–2313, Jul./Aug. 2022.
- [5] R. Geng, "FlowPinpoint: Localizing anomalies in cloud-client services for cloud providers," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3070–3084, Third Quarter, 2023.
- [6] F. Liu, J. Huang, and X. Wang, "Joint task offloading and resource allocation for device-edge-cloud collaboration with subtask dependencies," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3027–3039, Third Quarter, 2023.
- [7] V. Garg, S. Niranjana, V. Prybutok, T. Pohlen, and D. Gligor, "Drones in last-mile delivery: A systematic review on efficiency, accessibility, and sustainability," *Transp. Res. Part D Transport Environ.*, vol. 123, 2023, Art. no. 103831.
- [8] A. Hamdi, F. D. Salim, D. Y. Kim, A. G. Neiat, and A. Bouguettaya, "Drone-as-a-service composition under uncertainty," *IEEE Trans. Services Comput.*, vol. 15, no. 5, pp. 2685–2698, Sep./Oct. 2022.
- [9] J. Xu, X. Liu, X. Li, L. Zhang, J. Jin, and Y. Yang, "Energy-aware computation management strategy for smart logistic system with MEC," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8544–8559, Jun. 2022.
- [10] A. E. Eshratifar, M. S. Abrishami, and M. Pedram, "JointDNN: An efficient training and inference engine for intelligent mobile cloud computing services," *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 565–576, Feb. 2021.
- [11] A. E. Eshratifar, A. Esmaili, and M. Pedram, "BottleNet: A deep learning architecture for intelligent mobile cloud computing services," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Des.*, 2019, pp. 1–6.
- [12] B. Gao, Z. Zhou, F. Liu, and F. Xu, "Winning at the starting line: Joint network selection and service placement for mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1459–1467.
- [13] A. Hussain, J. Chun, and M. Khan, "A novel customer-centric methodology for optimal service selection (MOSS) in a cloud environment," *Future Gener. Comput. Syst.*, vol. 105, pp. 562–580, 2020.
- [14] G. J. Ibrahim, T. A. Rashid, and M. O. Akinsolu, "An energy efficient service composition mechanism using a hybrid meta-heuristic algorithm in a mobile cloud environment," *J. Parallel Distrib. Comput.*, vol. 143, pp. 77–87, 2020.
- [15] G. Zheng, H. Zhang, Y. Li, and L. Xi, "5G network-oriented hierarchical distributed cloud computing system resource optimization scheduling and allocation," *Comput. Commun.*, vol. 164, pp. 88–99, 2020.
- [16] P. Zhang, Y. Zhang, H. Dong, and H. Jin, "Mobility and dependence-aware QoS monitoring in mobile edge computing," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 1143–1157, Third Quarter, 2021.
- [17] Q. Peng et al., "Mobility-aware and migration-enabled online edge user allocation in mobile edge computing," in *Proc. IEEE Int. Conf. Web Serv.*, 2019, pp. 91–98.
- [18] B. Jiang, Q. He, X. Li, and H. Huang, "QoS control method based on SDN for mobile cloud service," in *Proc. IEEE 13th Int. Conf. Cloud Comput.*, 2020, pp. 275–283.
- [19] S. Ma, S. Song, J. Zhao, L. Zhai, and F. Yang, "Joint network selection and service placement based on particle swarm optimization for multi-access edge computing," *IEEE Access*, vol. 8, pp. 160 871–160 881, 2020.
- [20] M. Pomalo et al., "A data generator for cloud-edge vehicle communication in multi domain cellular networks," in *Proc. IEEE 7th Int. Conf. Internet Things Syst. Manage. Secur.*, 2020, pp. 1–8.
- [21] L. Chu, X. Li, J. Xu, A. G. Neiat, and X. Liu, "A holistic service provision strategy for drone-as-a-service in MEC-based UAV delivery," in *Proc. IEEE Int. Conf. Web Serv.*, 2021, pp. 669–674.
- [22] K. Dorling, J. Heinrichs, G. G. Messier, and S. Magierowski, "Vehicle routing problems for drone delivery," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 47, no. 1, pp. 70–85, Jan. 2017.
- [23] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, and L. Li, "Delay-aware and energy-efficient computation offloading in mobile-edge computing using deep reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 3, pp. 881–892, Sep. 2021.
- [24] M. Feng, M. Krunz, and W. Zhang, "Joint task partitioning and user association for latency minimization in mobile edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8108–8121, Aug. 2021.
- [25] J. Xu, X. Liu, X. Li, L. Zhang, and Y. Yang, "EXPRESS: An energy-efficient and secure framework for mobile edge computing and blockchain based smart systems," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2020, pp. 1283–1286.
- [26] R. Xie, Q. Tang, S. Qiao, H. Zhu, F. R. Yu, and T. Huang, "When serverless computing meets edge computing: Architecture, challenges, and open issues," *IEEE Wirel. Commun.*, vol. 28, no. 5, pp. 126–133, Oct. 2021.
- [27] X. Yang et al., "Temporal correlation characteristics of air-to-ground wireless channel with UAV wobble," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 10, pp. 10 702–10 715, Oct. 2023.
- [28] Z. Lv, D. Chen, R. Lou, and Q. Wang, "Intelligent edge computing based on machine learning for smart city," *Future Gener. Comput. Syst.*, vol. 115, pp. 90–99, 2021.

- [29] Z. Sun, B. Zhang, C. Gu, R. Xie, B. Qian, and H. Huang, "ET2FA: A hybrid heuristic algorithm for deadline-constrained workflow scheduling in cloud," *IEEE Trans. Services Comput.*, vol. 16, no. 3, pp. 1807–1821, May/Jun. 2023.
- [30] G. Yu, P. Chen, Z. Zheng, J. Zhang, X. Li, and Z. He, "FaaSDeiver: Cost-efficient and QoS-aware function delivery in computing continuum," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3332–3347, Sep./Oct. 2023.
- [31] L. Yin, J. Sun, J. Zhou, Z. Gu, and K. Li, "ECFA: An efficient convergent firefly algorithm for solving task scheduling problems in cloud-edge computing," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3280–3293, Sep./Oct. 2023.
- [32] Z. Liu, Q. Z. Sheng, X. Xu, D. Chu, and W. E. Zhang, "Context-aware and adaptive QoS prediction for mobile edge computing services," *IEEE Trans. Services Comput.*, vol. 15, no. 1, pp. 400–413, Jan./Feb. 2022.
- [33] H. Wu et al., "Mobility-aware service selection in mobile edge computing systems," in *Proc. IEEE Int. Conf. Web Serv.*, 2019, pp. 201–208.
- [34] I. Sittón-Candanedo, R. S. Alonso, J. M. Corchado, S. Rodríguez-González, and R. Casado-Vara, "A review of edge computing reference architectures and a new global edge proposal," *Future Gener. Comput. Syst.*, vol. 99, pp. 278–294, 2019.
- [35] Z. Zhou, J. Feng, Z. Chang, and X. Shen, "Energy-efficient edge computing service provisioning for vehicular networks: A consensus ADMM approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 5087–5099, May 2019.
- [36] C. Peng, X. Huang, Y. Wu, and J. Kang, "Constrained multi-objective optimization for UAV-enabled mobile edge computing: Offloading optimization and path planning," *IEEE Wireless Commun. Lett.*, vol. 11, no. 4, pp. 861–865, Apr. 2022.
- [37] S. Wang, Y. Zhao, L. Huang, J. Xu, and C.-H. Hsu, "QoS prediction for service recommendations in mobile edge computing," *J. Parallel Distrib. Comput.*, vol. 127, pp. 134–144, 2019.
- [38] B. Alkouz, A. Bouguettaya, and S. Mistry, "Swarm-based drone-as-a-service (SDaaS) for delivery," in *Proc. IEEE Int. Conf. Web Serv.*, 2020, pp. 441–448.



Azadeh Ghari Neiat received the PhD degree in computer science from RMIT University, Australia in 2017. She is a lecturer with the Information Technology, Deakin University, Australia. Her current research interests include Internet of Things (IoT), spatio-temporal data analysis, crowdsourcing/crowdsensing, Big Data mining, and machine learning with applications in the smart city, smart home, and recommender systems.



Lijun Chu received the bachelor's degree in Internet of Things engineering from the School of Computer and Information Engineering, Qufu Normal University, Qufu, Shandong, China, in 2018. She is currently working toward the master's degree with the School of Computer Science and Technology, Anhui University, Hefei, Anhui, China. Her current research interests include MEC, workflow system, service selection.



Xuejun Li (Member, IEEE) received the PhD degree in computer application technology from the School of Computer Science and Technology, Anhui University, Hefei, Anhui, China, in 2008. He is currently a full professor with the School of Computer Science and Technology, Anhui University, Hefei, Anhui, China. His major research interests include MEC, workflow systems, cloud computing, and intelligent software.



Jia Xu (Member, IEEE) received the bachelor's, master's, and PhD degrees in computer science and technology from the School of Computer Science and Technology, Anhui University, Hefei, Anhui, China, in 2014, 2017 and 2022, respectively. He was a software engineer focusing on industrial projects and solutions in iFLYTEK Company, Ltd from 2017–2018. From 2022 to 2024, he was a postdoctoral researcher with the School of Computer Science and Technology, Anhui University. He is currently a lecturer with the School of Internet, Anhui University. His current

research interests include MEC, workflow systems, cloud computing, and resource management.



Xiao Liu (Senior Member, IEEE) received the bachelor's and master's degrees in information management and information system from the School of Management, Hefei University of Technology, Hefei, China, in 2004 and 2007, respectively, and the PhD degree in computer science and software engineering from the Faculty of Information and Communication Technologies, Swinburne University of Technology, Melbourne, Australia, in 2011. He was an associate professor with the Software Engineering Institute, East China Normal University, Shanghai, China during

2013 to 2015. He is currently an associate professor with the School of Information Technology, Deakin University, Australia. His current research interests include software engineering, distributed computing, and data mining, with special interests in workflow systems, cloud/fog computing, and social networks.



Yun Yang (Senior Member, IEEE) received the PhD degree in computer science from the University of Queensland, Brisbane, QLD, Australia, in 1992. He is currently a full professor with the Department of Computing Technologies, Swinburne University of Technology, Melbourne, VIC, Australia. His current research interests include software technologies, cloud and edge computing, workflow systems, and service computing. He was associate editor of *IEEE Transactions on Cloud Computing*.